11.4.3 Precalculate positions

You can also precalculate positions and then apply transitions. If you have a complex algorithm that determines an element's new position, first go through the data array and calculate the new position. Then append the new position as data to the datapoint of the element. After you've done all your calculations, select and apply a transition based on the calculated new position. When you're calculating complex new positions and applying those calculated positions to a transition of a large selection of elements, you can overwhelm the browser and see jerky animations.

So, instead of

```
d3.selectAll(elements)
.transition()
.duration(1000)
.attr("x", newComplexPosition);
```

do this:

```
d3.selectAll(elements)
.each(function(d) {d.newX = newComplexPosition(d)});

d3.selectAll(elements)
.transition()
.duration(1000)
.attr("x", function(d) {return d.newX});
```

11.5 Summary

In this chapter, we looked at a few ways to deal with large datasets, and by necessity touched on methods for generating those datasets. Specifically, we looked at

- Generating random geodata
- Using the `.context` function of `d3.geo.path` to draw map features using canvas
- Using zoom's start and end functionality to render elements in canvas or SVG
- Generating random network data
- Drawing network lines in canvas
- Generating random xy data
- Creating an xy brush
- Highlighting selected features
- Building a quadtree
- Using a quadtree for optimized spatial search

In the next chapter (available as an online supplement), we'll focus on one area where performance tuning is important: data visualization on mobile. You'll see the built-in functionality in D3 for handling touch interfaces and spend time thinking about design principles for interactive data visualization on mobile.

If you want to grow your D3 skill set, I'd suggest starting with bl.ocksplorer (http://bl.ocksplorer.org/), which allows you to find examples of D3 code based on specific